REACT COURSE

Lesson 2 — Components & JSX

Functional Components • Reuse • Nesting • JSX • Fragments

Course	**React Development**
Lesson	**02**
Level	**Beginner**
Focus	**Components & JSX**

LearnSpace React Course

React Course — Lesson 2 | Components & JSX

1. Topic

Components & JSX

Functional Components

Component reuse

Component nesting

JSX attributes

JavaScript expressions in JSX

className

Fragments

Component organization

2. Definition

- Components are the building blocks of a React application. A component is an independent, reusable piece of user interface that has its own logic and appearance.
- Functional components are JavaScript functions that return JSX. They are the most common way to create components in modern React development.
- JSX is a syntax extension that allows writing HTML-like code inside JavaScript files. It makes React code readable and easy to understand.

3. Detailed Meaning

- React applications are built using components. Every part of the user interface can be divided into small, independent pieces called components.
- A functional component is simply a JavaScript function that returns JSX. The function name normally starts with a capital letter, which helps React understand that it is a component.
- Component reuse is one of the biggest advantages of React. Once a component is created, it can be used multiple times in the same application or in different applications with different data.

- Component nesting means using one component inside another component. For example, a Dashboard component can contain a Navbar component, a Sidebar component, and a Content component. This creates a tree-like structure of components.
- JSX attributes are used to provide information to elements. They work similarly to HTML attributes but many JSX property and event names use camelCase. For example, class becomes className and onclick becomes onClick.
- JavaScript expressions in JSX allow dynamic values to be displayed in the UI. Variables, calculations, function calls and conditional expressions can be placed inside curly braces.
- className is the JSX version of the HTML class attribute. It is used to apply CSS classes to elements.
- Fragments group multiple elements without adding an extra DOM element. The short syntax is <>...</>. This is useful when a component needs to return multiple sibling elements.
- Component organization means arranging component files and folders in a logical structure. Related components can be grouped together so that a project remains easy to understand and maintain.

4. Functional Components

A functional component is a normal JavaScript function that returns JSX. Modern React applications mainly use functional components.

```
function Welcome() {
return <h1>Welcome to React</h1>;
}

export default Welcome;
```

The function name is Welcome. Because it starts with a capital letter, React treats it as a component. The return statement contains the UI that should be displayed.

5. Component Naming

React component names should start with a capital letter. This makes a clear difference between a React component and a normal HTML element.

```
function CourseCard() {
return <div>React Course</div>;
}

function StudentProfile() {
return <div>Student Profile</div>;
}
```

Prefer:

```
<CourseCard />
<StudentProfile />
```

Avoid using lowercase names for custom React components:

```
<coursecard />
```

6. Component Reuse

Component reuse means creating a component once and using it multiple times. This avoids repeating the same UI code.

```
function CourseCard() {
return (
<div className="course-card">
<h2>React</h2>
<p>Learn React from basics.</p>
</div>
);
}

function App() {
return (
<div>
<CourseCard />
<CourseCard />
<CourseCard />
</div>
);
}
```

```
export default App;
```

The CourseCard component is written only once but rendered three times. In real applications, different data can be passed to each card using props.

7. Component Nesting

Component nesting means placing one component inside another component. It helps create a clear UI hierarchy.

```
function Navbar() {
return <nav>Navbar</nav>;
}

function Sidebar() {
return <aside>Sidebar</aside>;
}

function Dashboard() {
return (
<div>
<Navbar />
<Sidebar />
<main>Dashboard Content</main>
</div>
);
}
```

```
export default Dashboard;
```

The structure can be understood as:

```
Dashboard
├── Navbar
├── Sidebar
└── Main Content
```

8. JSX Attributes

JSX attributes provide additional information to HTML elements or React components. They look similar to HTML attributes, but JSX follows JavaScript naming conventions in many cases.

```
function App() {
return (
<div className="container">
<input type="text" placeholder="Enter your name" />
<button onClick={() => alert("Hello")}>
Click Me
</button>
</div>
);
}
```

Common JSX Attributes

HTML	JSX	Purpose

class	className	Apply CSS class
onclick	onClick	Handle click event
tabindex	tabIndex	Control keyboard tab order
for	htmlFor	Connect label with input

9. JavaScript Expressions in JSX

Curly braces { } are used to place JavaScript expressions inside JSX. This makes it possible to display dynamic data.

```
function App() {
const name = "John";
const age = 21;
const price = 500;

return (
<div>
<h1>Hello {name}</h1>
<p>Age: {age}</p>
<p>Total: ₹{price * 2}</p>
</div>
);
}
```

Expressions vs Statements

An expression produces a value and can be used inside JSX. Examples include variables, calculations, function calls and ternary expressions.

```
{name}
{age + 1}
{price * quantity}
{getMessage()}
{isLoggedIn ? "Logout" : "Login"}
```

Statements such as if blocks and for loops cannot normally be written directly inside JSX. For these cases, use suitable JavaScript logic outside JSX or expressions such as the ternary operator and map.

10. className

In HTML, CSS classes are assigned using class. In JSX, className is used because class has a special meaning in JavaScript.

```
function App() {
return (
<div className="container">
<h1 className="title">React Course</h1>
<p className="description">
Learn React step by step.
</p>
</div>
```

```
);
}
```

Example CSS:

```
.container {
padding: 20px;
}

.title {
font-size: 30px;
}

.description {
line-height: 1.6;
}
```

React Course — Lesson 2 | Components & JSX

11. Fragments

A Fragment lets a component return multiple elements without adding an extra HTML element to the DOM.

Without Fragment

```
function App() {
return (
<div>
<h1>React</h1>
<p>Learn Components</p>
```

```
</div>
);
}
```

With Fragment

```
function App() {
return (
<>
<h1>React</h1>
<p>Learn Components</p>
</>
);
}
```

The short Fragment syntax <>...</> is useful when an extra div is not needed.

Named Fragment

```
import { Fragment } from "react";

function App() {
return (
<Fragment>
<h1>React</h1>
<p>Components</p>
</Fragment>
);
}
```

12. Component Organization

As a React project grows, keeping every component in one file becomes difficult. Components should be separated into files and organized into folders.

```
src/
├── components/
│   ├── Navbar.jsx
│   ├── Footer.jsx
│   ├── CourseCard.jsx
│   └── Button.jsx
│
├── pages/
│   ├── Home.jsx
│   ├── Courses.jsx
│   └── Dashboard.jsx
│
├── App.jsx
└── main.jsx
```

Small reusable UI components can be placed inside components. Page-level components can be placed inside pages. The exact folder structure can change depending on the size and requirements of the project.

13. Importing and Exporting Components

A component can be exported from one file and imported into another file.

```
// CourseCard.jsx
function CourseCard() {
return <div>React Course</div>;
}

export default CourseCard;
```

```
// App.jsx
import CourseCard from "./CourseCard";

function App() {
return (
<div>
<CourseCard />
</div>
);
}

export default App;
```

This allows components to remain in separate files while still being combined to create the complete application.

14. Passing Data to Reusable Components

A reusable component becomes more useful when it can display different data. Data can be passed to a component using props. Props are covered in detail in the next lesson.

```
function CourseCard({ name }) {
return <h2>{name}</h2>;
}

function App() {
return (
<div>
<CourseCard name="React" />
<CourseCard name="Node.js" />
<CourseCard name="MongoDB" />
</div>
);
}
```

Here the same CourseCard component is reused three times with different values.

15. Example

A React example demonstrating component reuse and nesting with multiple child components:

```
function StudentCard({ name, course, grade }) {
return (
<div className="card">
<h2>{name}</h2>
<p>Course: {course}</p>
<p>Grade: {grade}</p>
</div>
```

```
);
}

function StudentList() {
const students = [
{ name: "John", course: "React", grade: "A" },
{ name: "Jane", course: "Node.js", grade: "B+" },
{ name: "Mike", course: "MongoDB", grade: "A-" }
];

return (
<div>
<h1>Student Dashboard</h1>

{students.map((student, index) => (
<StudentCard
key={index}
name={student.name}
course={student.course}
grade={student.grade}
/>
))}
</div>
);
}
```

16. Code Example

```
import { Fragment } from "react";

function Header({ title }) {
return <h1 className="header-title">{title}</h1>;
}

function Footer({ year }) {
return <footer className="footer">© {year} My
Website</footer>;
}

function App() {
const courses = ["React", "Node.js", "MongoDB"];

return (
<Fragment>
<Header title="Welcome to LMS" />

<main>
<h2>Available Courses</h2>

<ul>
{courses.map((course, index) => (
<li key={index}>{course}</li>
))}
```

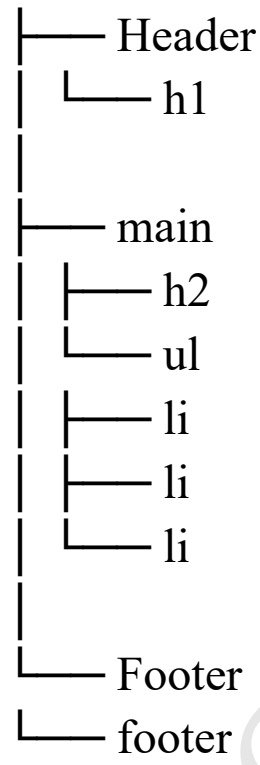
```
</ul>
</main>

<Footer year={2024} />
</Fragment>
);
}

export default App;
```

17. Code Explanation

- Header component: Header is a separate functional component that receives a title and displays it.
- Footer component: Footer is another reusable component that receives a year value.
- App component: App works as the parent component and combines Header, main content and Footer.
- Component nesting: Header and Footer are placed inside App, creating a parent-child component structure.
- className: header-title and footer are CSS class names applied using className.
- courses array: The courses variable stores three course names.
- map(): The map method creates one list item for every course.
- key: The key helps React identify list items when rendering a list.

- Fragment: Fragment groups the top-level elements without adding an unnecessary wrapper element.
- JavaScript expressions: Values such as title, year and course are inserted into JSX using curly braces.

18. Component Tree



Thinking about an application as a component tree makes large React projects easier to understand. Each component can have a specific responsibility.

19. Component Best Practices

- Keep components small and focused on one main responsibility.
- Use meaningful component names such as CourseCard or StudentProfile.
- Reuse components instead of copying the same UI code.
- Keep related components in an organized folder structure.
- Use props when the same component needs different data.
- Avoid making one component unnecessarily large.
- Use className for CSS classes in JSX.
- Use fragments when an extra wrapper element is not required.

20. Common Mistakes

- Lowercase custom component: <courseCard /> instead of <CourseCard />.
- Missing closing tag: without or a self-closing form.
- Using class: Use className instead of class in JSX.
- Forgetting curly braces: Use {variable} when inserting a JavaScript value into JSX.
- Repeating UI code: Create a reusable component instead of copying the same markup.
- Unnecessary wrapper: Use a Fragment when an extra div is not required.

21. Summary

- Components are the building blocks of React applications.
- Functional components are JavaScript functions that return JSX.
- Component names should start with a capital letter.
- A component can be reused multiple times.
- Components can be nested inside other components.
- JSX attributes are similar to HTML attributes but follow JSX naming rules.
- JavaScript expressions are written inside curly braces.
- className is used instead of class in JSX.
- Fragments group elements without adding an extra DOM node.
- Components should be organized into logical files and folders.
- Props can be used to pass different data to reusable components.

22. Quick Revision Questions

1. What is a functional component?

2. Why should React component names start with a capital letter?

3. What is component reuse?

4. What is component nesting?

5. Why is className used instead of class?

6. How are JavaScript expressions written in JSX?
7. What is a Fragment?
8. What is the purpose of the map method in a React list?
9. Why are components separated into different files?
10. How can the same component display different data?